GraphHunter

Reporte interno de investigación

Lucas Sotomayor
Base4Security

13 de mayo de 2026

Índice general

Resumen **4**

1. Introducción **5**
1.1. Motivación . . . 5
1.2. Problema . . . 5
1.3. Alcance y no-alcance . . . 5
1.4. Contribuciones . . . 5
1.5. Organización . . . 6

2. Trabajo relacionado **7**
2.1. Lenguajes de consulta en SIEM . . . 7
2.2. Threat hunting sobre grafos de procedencia . . . 7
2.3. Detección de anomalías sobre grafos heterogéneos . . . 7
2.4. Superficies agénticas . . . 8
2.5. Compiladores asistidos por LLM . . . 8

3. Arquitectura: capas de cizalla **9**
3.1. Idea central . . . 9
3.2. Crates y lenguajes . . . 9
3.3. Contraste con SOLID uniforme . . . 10
3.4. Propiedades verificables . . . 10
3.5. Relación con los otros libros . . . 10

4. Matcher: DFS puro-Rust + intersección SIMD AVX2 **11**
4.1. Problema . . . 11
4.2. DFS iterativo en Rust . . . 11
4.2.1. Cache SoA de tipos de entidad . . . 12
4.2.2. Software prefetch L1 . . . 12
4.3. Primitiva SIMD: intersección sorted-set $u32$ . . . 13
4.3.1. Algoritmo AVX2 *broadcast & compare* . . . 13
4.3.2. Gallop seek para LFTJ . . . 14
4.4. Equivalencia y verificación . . . 14
4.5. Throughput medido . . . 15
4.6. Dominios de aplicación . . . 15
4.7. Status badge en UI . . . 15

5. DFS temporal guiado por score **16**
5.1. Rol dentro del pipeline . . . 16
5.2. Hipótesis como forma de patrón . . . 16
5.3. Algoritmo . . . 17
5.4. Propiedades y complejidad . . . 17
5.5. ¿Por qué DFS y no Ullmann/VF2? . . . 18

6. Scoring compuesto **19**
6.1. Motivación . . . 19
6.2. Los cinco componentes . . . 19
6.2.1. Entity rarity . . . 19
6.2.2. Edge rarity . . . 19
6.2.3. Neighborhood concentration . . . 20
6.2.4. Temporal novelty . . . 20
6.2.5. GNN threat . . . 20
6.3. Composición . . . 20
6.4. Propiedades numéricas . . . 20
6.5. Explicación por path . . . 20

7. Decisión arquitectónica: eliminación de la frontera FFI **21**
7.1. Diseño original (resumen breve) . . . 21
7.2. Hallazgo empírico que precipitó la eliminación . . . 21
7.3. Decisión . . . 22
7.4. Forma del crate residual . . . 23
7.5. Consecuencia arquitectónica . . . 23
7.6. Pérdida controlada . . . 23

8. Model gate: GNN experimental opcional **25**
8.1. El dilema del componente ML . . . 25
8.2. Contrato . . . 25
8.3. Arquitectura del modelo v2 . . . 26
8.4. Separación layer-wise . . . 26
8.5. Honestidad epistémica . . . 26
8.6. Extensiones previstas . . . 26

9. Ingestor agéntico: LLM-as-compiler offline y drift-as-signal **27**
9.1. Problema . . . 27
9.2. Decisión: tres carriles . . . 27
9.3. LLM como compilador, no como traductor . . . 27
9.4. Drift como señal, no como error . . . 28
9.5. Propiedades formales . . . 28
9.6. Trabajo relacionado . . . 29
9.7. Limitaciones y trabajo futuro . . . 29

10. Evaluación **30**
10.1. Benchmarks de performance . . . 30
10.2. Property-based tests . . . 31
10.3. Smoke end-to-end manual . . . 31
10.4. Métricas del proyecto . . . 32
10.5. Comparación pre/post shearing-layers . . . 32

11. Limitaciones **33**
11.1. Arquitectura . . . 33
11.2. Modelo ML . . . 33
11.3. Tests . . . 34
11.4. Evaluación . . . 34
11.5. Superficies . . . 34
11.6. Operación . . . 35

12. Mapa *claim* → evidencia **36**
12.1. Tabla principal . . . 37
12.2. Reclasificación de contribuciones . . . 38
12.3. Roadmap 30/60/90 (desde 2026-04-24) . . . 38
12.4. Regla de cierre . . . 38

13. Conclusiones **39**
13.1. Síntesis . . . 39
13.2. Qué aprendimos del rediseño . . . 39
13.3. Trabajo futuro . . . 40
13.4. Cierre . . . 40

Resumen

La caza de amenazas sobre grafos heterogéneos construidos a partir de logs de seguridad choca contra tres obstáculos recurrentes: (i) ingesta frágil frente a formatos nuevos o mutantes, (ii) costo combinatorio del *matching* de patrones en grafos grandes con múltiples tipos de entidades y aristas, (iii) señales de amenaza mezcladas con ruido operacional sin un esquema de priorización estable. Los SIEM tradicionales responden sobre tablas con lenguajes orientados a texto; el analista pierde la estructura relacional del incidente en el momento mismo de formular la pregunta.

Este reporte describe GraphHunter, una arquitectura diseñada para atacar los tres obstáculos por separado. La contribución principal es una descomposición en *shearing layers* que aísla tres capas con velocidades de cambio distintas: un núcleo estable con el motor de grafo, el matcher DFS Rust con primitiva SIMD AVX2 y el gate de ML; una plataforma con superficies (HTTP, MCP, DSL, parsers, canonical OCSF); y un conjunto de aplicaciones delgadas. Sobre esa descomposición presentamos: un matcher de patrones DFS iterativo con intersección AVX2 de posting-lists (`simd\_rust.rs`, runtime dispatch) cuya equivalencia AVX2/escalar se verifica por property-testing; un DFS temporal guiado por score para la fase de *refinement*; un compositor de anomalía de cinco componentes con pesos operables; una decisión arquitectónica documentada de *eliminar la frontera FFI* sobre `libgraphmatch` (motor C++ AVX2) el 2026-05-07 al demostrar que el DFS Rust era 3-25× más rápido; un *model gate* que hace opcional al GNN de amenazas sin degradar el camino base; y un ingestor agéntico de tres carriles (VRL compilado, LLM-como-compilador offline, drift-como-señal) que combina determinismo productivo con adaptación offline gobernada. La evaluación incluye benchmarks de latencia y throughput, unit tests de equivalencia AVX2/escalar y paridad LFTJ *naive* = *LFTJ* = *LFTJ+SIMD*, y un smoke manual end-to-end sobre datasets sintéticos. Concluimos con limitaciones abiertas: dos aristas residuales `core`→`platform` y la ausencia de un modelo GNN entrenado sobre FortiAnalyzer producción.

El libro complementario *Algoritmos* (`docs/algoritmos/algoritmos.pdf`) profundiza en el diseño algorítmico del *matcher* y los índices auxiliares con diagramas pedagógicos: el *stack* de operadores físicos (DFS, LFTJ-K4, Yannakakis chain, Yannakakis tree-walk), la *lazy build* y el *sparse mmap-spill* de NLF y k-hop reachability, y el plano de obra de las stages futuras de la *streaming layer*. El apéndice E consolida el *mapa interno* del motor.

Capítulo 1

Introducción

1.1. Motivación

Un analista que investiga un incidente de seguridad raramente se hace preguntas tabulares. Las preguntas tienen forma relacional: *¿qué usuarios autenticaron desde la IP que conectó al host comprometido, dentro de una ventana de una hora?*. El SIEM entiende *usuario*, *IP* y *host* como columnas y la pregunta, como una serie de joins textuales. La estructura relacional del incidente vive en la cabeza del analista. GraphHunter la explicita y la mueve al código.

1.2. Problema

Formalizamos el problema en tres subproblemas con propiedades distintas:

1. **Ingesta heterogénea**. Datos provienen de fuentes con esquemas distintos (EDR, firewalls, proxies, IAM, cloud audit). Los formatos mutan en el tiempo. El objetivo es un grafo canonical homogéneo sin perder la fidelidad del origen.

2. **Matching estructural**. Dado un grafo $G = (V, E)$ con nodos y aristas tipados y una hipótesis (patrón en un DSL), enumerar *paths* instancia del patrón con cotas temporales y estructurales. El espacio es combinatorio: naïf explota con la multiplicación de vecindarios.

3. **Priorización**. El *matching* produce muchos paths; la mayoría son actividad legítima. Un analista puede triar orden de decenas por turno. El objetivo es un score compuesto, interpretable y con pesos operables.

1.3. Alcance y no-alcance

Este reporte describe las decisiones de diseño que fijan la forma del sistema actual (*post-shearing-layers*) y el razonamiento detrás de cada una, con evaluación cuantitativa donde hay números y honesta donde no. No es un paper académico con peer review; es un reporte interno para consumo del equipo y eventualmente de auditores técnicos externos.

Quedan fuera de alcance: el motor de UI (`apps/tauri`), las políticas organizacionales que usan GraphHunter como herramienta de triage, y la operación del *stack* de observabilidad sobre el que corre. Esos temas tienen su propio lugar en el *manual* y en la *especificación*.

1.4. Contribuciones

Este trabajo aporta:

1. Una descomposición en *shearing layers* (cap. 3) que reduce acoplamiento y permite evolucionar superficies sin tocar el núcleo.

2. Un matcher DFS iterativo Rust con primitiva SIMD AVX2 de intersección sorted-set (cap. 4) con equivalencia AVX2/escalar verificada por unit tests y paridad LFTJ *naive* = *LFTJ* = *LFTJ+SIMD*.

3. Un DFS temporal guiado por score para el *refinement* acotado por k-simplicity (cap. 5).

4. Un composer de anomalía de cinco componentes con pesos operables (cap. 6).

5. Una decisión arquitectónica documentada de *eliminar la frontera FFI* sobre el motor C++ `libgraphmatch` al demostrarse 3-25× inferior al DFS puro-Rust (cap. 7; 2026-05-07).

6. Un *model gate* que vuelve opcional al clasificador GNN (cap. 8).

7. Un ingestor de tres carriles con LLM-como-compilador offline y drift-como-señal (cap. 9).

1.5. Organización

El capítulo 2 revisa enfoques previos. El 3 resume la arquitectura global. Los capítulos 4–8 desarrollan las contribuciones técnicas. El 9 cubre el ingestor agéntico. El 10 presenta evaluación. El 11 discute limitaciones abiertas y el 13 concluye.

Capítulo 2

Trabajo relacionado

GraphHunter se ubica en la intersección de tres líneas: lenguajes de consulta de SIEM, *threat hunting* sobre grafos de procedencia, y detección de anomalías sobre grafos heterogéneos. Este capítulo discute representativos de cada línea y sitúa las decisiones de diseño propias respecto a ellos.

2.1. Lenguajes de consulta en SIEM

Los SIEM contemporáneos ofrecen lenguajes orientados a tablas con extensiones para series de tiempo: Splunk SPL, Azure KQL, Elastic EQL. La expresividad es suficiente para *detection rules* y búsquedas textuales con agregaciones. La estructura relacional se modela con joins textuales sucesivos, legibles en ejemplos pequeños, frágiles en investigaciones de varios hops.

La diferencia conceptual es *dónde vive* la estructura del incidente. En SIEM, vive en la redacción de la query (‘... | join ... | join ...‘) que el analista debe mantener coherente. En GraphHunter, vive en el grafo canonical y la hipótesis DSL (cap. 5) sólo describe el patrón; la materialización de paths la hace el motor.

2.2. Threat hunting sobre grafos de procedencia

HOLMES [1] y Poirot [2] inauguraron la línea de construir grafos causales a partir de eventos del sistema operativo (fork/exec, open, connect) y emparejarlos contra patrones derivados de ATT&CK. RapSheet y trabajos posteriores refinan la forma de los patrones y las heurísticas de ranking. El target común es el grafo de procedencia del host.

GraphHunter difiere en tres ejes respecto a esta línea:

1. **Fuente de datos**. En lugar de grafo causal del kernel, trabaja sobre un grafo canonical construido desde logs heterogéneos (OCSF v1.4). Permite razonar entre hosts y entre dominios (red + IAM + cloud audit), no sólo dentro de un host.

2. **Lenguaje**. El DSL es externo y composicional (cap. 5); HOLMES/Poirot embeben los patrones en reglas internas. Permite a un analista escribir hipótesis nuevas sin tocar código.

3. **Scoring**. Componemos cinco señales de anomalía operables (cap. 6). La línea HOLMES tiende a scoring binario (match/no-match) con umbrales ocultos.

2.3. Detección de anomalías sobre grafos heterogéneos

Una línea paralela emplea representaciones aprendidas (DeepWalk [3], embeddings GNN) para detectar anomalías sin patrón explícito. Las arquitecturas con GCN sobre grafos de logs

son efectivas cuando existe volumen de datos etiquetados. Tienen dos costos operativos bien documentados: distribución shift entre entornos (un modelo entrenado en un banco no generaliza a una telco) y dificultad de dar explicaciones al analista.

GraphHunter usa anomalía como *señal composicional*: los cinco componentes (entity rarity, edge rarity, neighborhood concentration, temporal novelty, gnn threat) son interpretables por separado. El componente GNN está tras un *model gate* (cap. 8) y no bloquea el uso del sistema sin modelo entrenado. Esto es deliberado: el sistema debe funcionar el primer día en un cliente sin dataset propio.

2.4. Superficies agénticas

La interacción con asistentes LLM via Model Context Protocol (MCP) es un espacio todavía en estandarización. Los trabajos publicados se enfocan en servidores MCP genéricos sobre bases de datos tabulares o APIs REST. GraphHunter expone 55 tools agrupadas en 10 dominios semánticos (search, graph, hunt, catalog, explorer, tags, notes, export, ingest, sessions) permitiendo a Claude Code o Cursor conducir investigaciones completas.

2.5. Compiladores asistidos por LLM

El ingestor agéntico (cap. 9) explota una idea conocida en *program synthesis*: el LLM propone un artefacto de *configuración*, no el código ejecutable. El código lo produce un compilador determinístico. Esa separación preserva la propiedad de determinismo productivo que el LLM por sí sólo no da. El patrón es similar al usado en Copilot-for-SQL y variantes: el LLM sugiere, un validador rechaza lo inválido, el humano ratifica.

Capítulo 3

Arquitectura: capas de cizalla

3.1. Idea central

Shearing layers [4] organiza un sistema por velocidades de cambio. Lo que cambia rápido (pieles, superficies, integraciones) no debe acoplar a lo que cambia lento (esqueletos, motores). En software, la consecuencia práctica es dirigir todas las dependencias *desde* lo rápido *hacia* lo lento, y prohibir dependencias en sentido contrario. Una arista núcleo→plataforma, si existe, es una fuente de fricción estructural: el núcleo se ve forzado a moverse a la velocidad de la superficie.

GraphHunter adopta tres capas con esta disciplina:

1. **Core** ("esqueleto"): tipos del grafo, operaciones estructurales, scoring, matcher DFS Rust con primitiva SIMD AVX2, clasificador GNN. Cambia por meses.
2. **Platform** ("carne"): DSL, parsers, sources de ingesta, canonical OCSF, servidor HTTP, MCP TypeScript, servicios compartidos (catálogo, DLQ, drift monitor, mapping library). Cambia por semanas.
3. **Apps** ("piel"): Tauri desktop, CLI. Cambia por días.

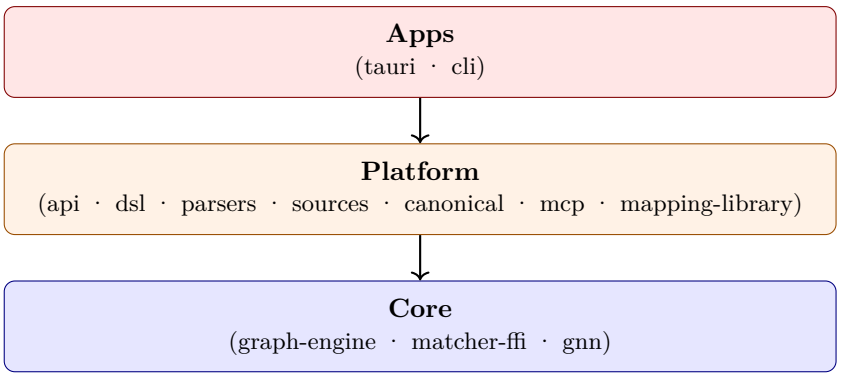


Figura 3.1: Las tres capas. Todas las aristas de dependencia apuntan hacia abajo (piel→carne→esqueleto). No hay ciclos.

3.2. Crates y lenguajes

El sistema está compuesto por 14 *crates* Rust y un servidor MCP en TypeScript. La elección de lenguajes sigue la lógica de shearing:

- **Rust** en todo el sistema: tipos fuertes, zero-cost abstractions, seguridad sin recolector. Es el tejido por defecto. La intersección SIMD AVX2 vive en `core/graph-engine/src/simd\_rust.rs` con runtime dispatch.
- **TypeScript** sólo para la superficie MCP: el protocolo tiene SDK oficial en TS y la audiencia es asistentes LLM con tooling JS.

Nota 2026-05-07: hasta esa fecha existía además una biblioteca C++ (`libgraphmatch`) con intrínsecas AVX2 para el matcher. Fue eliminada tras demostrarse que el DFS puro-Rust de `core/graph-engine` la supera en 3-25× sobre cada workload medido.

3.3. Contraste con SOLID uniforme

Una crítica posible es: *"ya existe SOLID, ¿para qué otro modelo?"*. SOLID es un conjunto de reglas micro, aplicables a clases individuales. Shearing layers es una regla macro: aplica a *módulos de alto nivel*. Son complementarios. La diferencia relevante es la dirección de la presión de diseño: SOLID empuja a abstraer *por contrato*; shearing empuja a separar *por velocidad de cambio*. Un sistema bien SOLID puede seguir teniendo dependencias núcleo→superficie; shearing prohibe eso explícitamente.

3.4. Propiedades verificables

La descomposición se validó con tres propiedades:

1. **Cero ciclos**. Verificado con `cargo-modules`/`cargo-deps` y `cargo check` sobre el workspace. Un ciclo rompería la capacidad de aislar lo que cambia.
2. **Sin frontera FFI activa**. Tras la eliminación de `libgraphmatch` (2026-05-07, cap. 7) el árbol es 100 % Rust; `matcher-ffi` sobrevive como *thin forwarder* Rust puro ≈30 LOC. El único `extern "C"` residual es a `nauty-Traces-sys` bajo `cfg(unix) + feature symmetry-nauty`.
3. **Canonical API *sealed***. Los tipos que cruzan la superficie HTTP/MCP se definen en una capa de DTOs; las mutaciones del grafo canonical no escapan.

Las dos aristas residuales `core`→`platform` (ver cap. 11) no son ciclos pero invierten la dirección deseada. Son deuda reconocida.

3.5. Relación con los otros libros

Este capítulo resume lo mínimo para situar las contribuciones técnicas que siguen. La versión detallada del modelo, con el inventario completo de crates y seis ADRs, vive en el libro *Arquitectura* de la documentación.

Capítulo 4

Matcher: DFS puro-Rust + intersección SIMD AVX2

Estado (2026-05-12). Este capítulo describe la arquitectura actual *del matcher tras la eliminación de* `libgraphmatch` *(motor C++ AVX2 integrado por FFI) el 2026-05-07. El DFS vive enteramente en Rust en* `core/graph-engine`*; la intersección sorted-set AVX2 sobrevive como primitiva en* `core/graph-engine/src/simd\_rust.rs` *con runtime dispatch y sin feature flag de build. Ver* `docs/perf/2026-05-07-post-libgraphmatch-removal.md` *para los números pre/post-migración.*

4.1. Problema

Dado un grafo $G = (V, E)$ con $|V| \approx 10^5$ y una hipótesis que describe un patrón de longitud 2–4 hops con restricciones de tipo por nodo y arista, el objetivo es enumerar instancias del patrón. Dos primitivas dominan el costo:

1. **Expansión de frontera por DFS temporal**: para cada path parcial, iterar las aristas salientes filtrando por tipo y ventana temporal.
2. **Intersección de posting-lists ordenadas** (`u32` sorted-set), invocada por la familia LFTJ *worst-case-optimal* para triángulos, K_4, C_6 y K_5.

Una implementación escalar pura sufre cache misses sobre los tags de tipo de entidad (un `HashMap` probe por arista evaluada) y branch mispredictions en la intersección. El objetivo del diseño actual es eliminar ambos sin salirse del toolchain Rust.

4.2. DFS iterativo en Rust

El matcher es un DFS *iterativo* (sin recursión) con stack explícito de `AnomalyFrame`. Cada frame mantiene el path parcial, el acumulador de anomaly score y el iterador de aristas locales. La entrada pública es:

```
impl GraphHunter {
    pub fn search_temporal_pattern(
        &self,
        hypothesis: &Hypothesis,
        time_window: Option<(i64, i64)>,
        max_results: Option<usize>,
    ) -> Result<(Vec<HuntResult>, bool), GraphError> { ... }

    pub fn search_temporal_pattern_smart(
```

```
        &self,
        hypothesis: &Hypothesis,
        time_window: Option<(i64, i64)>,
        top_k: usize,
    ) -> Result<(Vec<HuntResult>, bool), GraphError> { ... }
}
```

`search\_temporal\_pattern` devuelve hasta `cap` resultados (default 10^4) y un flag `has\_more` que indica saturación. `search\_temporal\_pattern\_smart` aplica top-k con poda por anomaly scorer, fundiendo min-heaps por-chunk en paralelo (rayon) y descartando paths por debajo de un umbral de percentil.

4.2.1. Cache SoA de tipos de entidad

El cuello de botella original era el chequeo de tipo de la entidad destino. En lugar de mirar el `HashMap<StrId, Entity>`, el matcher mantiene un `Vec<u8>` paralelo:

```
// core/graph-engine/src/graph.rs:99-110
/// SoA hot-path index: entity_type_tags[sid.index()] holds the u8
/// type tag for that entity (0..=252) or TAG_DEAD (253) if removed.
pub(crate) entity_type_tags: Vec<u8>,
```

El layout *Structure-of-Arrays* mete el tipo de cada destino posible en una línea de cache contigua. El probe se reduce a `tags[sid.index()] == required\_tag\_u8`, cero punteros indirectos. Se popula al construir el grafo y nunca se realoja mientras el grafo no se modifique.

4.2.2. Software prefetch L1

Sobre x86_64 el inner loop pre-prefetcha la siguiente línea de `entity\_type\_tags` con `\_mm\_prefetch`:

```
// core/graph-engine/src/graph.rs:1690-1711
// B4: software prefetch of entity_type_tags for dest_sids
// ahead of the inner loop. Hides L1 miss latency.
#[cfg(target_arch = "x86_64")]
{
    const LOOKAHEAD: usize = 4;
    if idx + LOOKAHEAD < edges_local.len() {
        let probe_dst =
            edges_local[idx + LOOKAHEAD].dest_sid.index();
        if probe_dst < self.entity_type_tags.len() {
            unsafe {
                std::arch::x86_64::_mm_prefetch(
                    &self.entity_type_tags[probe_dst]
                        as *const u8 as *const i8,
                    std::arch::x86_64::_MM_HINT_T0,
                );
            }
        }
    }
}
```

Un lookahead de 4 aristas oculta la latencia L1 detrás del trabajo de la iteración actual. Está bajo `cfg(target\_arch = "x86\_64")` porque `\_mm\_prefetch` es un intrínseco x86; los otros targets compilan sin él sin warnings.

4.3. Primitiva SIMD: intersección sorted-set $u32$

La intersección sobrevive como una primitiva Rust pura en `core/graph-engine/src/simd\_rust.rs`. La firma pública:

```
pub fn intersect_sorted_u32_count(a: &[u32], b: &[u32]) -> usize;
pub fn intersect_sorted_u32_collect(
    a: &[u32], b: &[u32], out: &mut Vec<u32>);
pub fn gallop_ge_u32(s: &[u32], target: u32) -> usize;
pub fn isa_label() -> &'static str;  // "avx2" | "scalar"
```

El dispatcher es totalmente runtime: ningún feature de Cargo controla el codepath. Sobre x86_64 con AVX2 reportada por el host se ejecuta el body AVX2; en cualquier otro caso (o slices con < 32 elementos donde el setup vectorial no paga) se ejecuta el merge escalar.

```
// core/graph-engine/src/simd_rust.rs:32-45
pub fn intersect_sorted_u32_count(a: &[u32], b: &[u32]) -> usize {
    if a.len() < 32 || b.len() < 32 {
        return scalar_intersect_count(a, b);
    }
    #[cfg(target_arch = "x86_64")]
    {
        if is_x86_feature_detected!("avx2") {
            // SAFETY: AVX2 just verified at runtime.
            return unsafe { avx2_intersect_count(a, b) };
        }
    }
    scalar_intersect_count(a, b)
}
```

4.3.1. Algoritmo AVX2 *broadcast & compare*

El kernel itera el lado más corto y, por cada sv, hace broadcast a 8 lanes y compara contra una ventana de 8 elementos del lado más largo:

Algorithm 4.1: Intersección AVX2 broadcast-and-compare

Input: A, B listas u32 ordenadas y deduplicadas
Output: $\#(A \cap B)$

$(s, l) \leftarrow (A, B)$ si $|A| \leq |B|$, sino (B, A);
$j \leftarrow 0$; count $\leftarrow 0$;
foreach $sv \in s$ **do**
  while $j + 8 \leq |l|$ **do**
    chunk $\leftarrow$ `_mm256_loadu_si256`$(l + j)$;
    needle $\leftarrow$ `_mm256_set1_epi32`(sv);
    cmp $\leftarrow$ `_mm256_cmpeq_epi32`(chunk, needle);
    mask $\leftarrow$ `_mm256_movemask_epi8`(cmp);
    if mask $\neq 0$ **then**
      count+=1; **break**;
    end
    if $l[j + 7] < sv$ **then**
      j+=8; **continue**;
    end
    break;
  end
  ▷ `cola escalar para tail < 8`
end
return count

`\_mm256\_cmpeq\_epi32` produce máscaras por lane que `\_mm256\_movemask\_epi8` comprime a 32 bits; un único test `!= 0` basta para decidir si hubo match en la ventana. La estrategia es $O(|s| \cdot |l|/8)$ en el peor caso pero compite bien cuando los tamaños son skew o cuando los datos viven en L1.

4.3.2. Gallop seek para LFTJ

`gallop\_ge\_u32` es la búsqueda exponencial usada por el inner loop de `CsrTrieIter::seek`: duplica el step hasta pasar el target, luego `binary\_search` sobre la ventana acotada. Es estrictamente más barato que un `binary\_search` desnudo cuando el target está cerca del cursor actual (el patrón WCOJ donde cada iterador sólo avanza ligeramente sobre la key del vecino).

4.4. Equivalencia y verificación

La intersección AVX2 se verifica contra dos referencias:

1. **Reference intersect** sobre `BTreeSet` en los 15 tests de `core/graph-engine/src/simd\_rust.rs` (`cargo test -p graph\_hunter\_core --lib simd\_rust`): empty, disjoint, identical, partial-overlap, scalar-vs-AVX2 sobre slices generados con LCG, simetría, boundary en el último lane de una ventana, tail past last full window.

2. **Parity LFTJ** en `core/graph-engine/src/lftj.rs`: 16 tests pasan, incluyendo paridad triple naive = LFTJ = LFTJ+SIMD sobre conteo de triángulos.

El bench `core/matcher-ffi/src/bin/bench\_dfs.rs` *(que sobrevive como cliente del engine, no como FFI a C++)* mide `search\_temporal\_pattern` y `\_smart` sobre grafos Barabási-Albert de 5k y 20k nodos para los patrones `spawn\_chain`, `lateral` y `any\_chain` de longitud 2-4 hops.

4.5. Throughput medido

Comparación pre/post eliminación del FFI C++ sobre el mismo host (Ryzen 5 PRO 5650U, Zen 3, AVX2 sin AVX-512), *big graph* de 20k nodos y ≈ 160k aristas:

Patrón	pre (C++ FFI, ms)	post (Rust DFS, ms)	delta
spawn_chain L=2	0.174	0.133	−24 %
spawn_chain L=3	0.132	0.095	−28 %
lateral L=3	0.137	0.111	−19 %
lateral L=4	0.109	0.083	−24 %
any_chain L=2	0.955	0.756	−21 %
any_chain L=3	1.089	0.807	−26 %
any_chain L=4	1.151	0.878	−24 %

Tabla 4.1: `search\_temporal\_pattern` cap-saturado a 10^4 resultados, antes y después de la migración. El DFS Rust gana entre 19 y 28 % sobre cada patrón del *big graph*, el tamaño operativamente relevante.

Frente al motor C++ FFI medido aisladamente (con `build\_gm\_graph` + `run\_simd\_search`), el DFS Rust ya ganaba por 3.4-19.4× en la misma plataforma; ver `docs/perf/2026-05-07-pre-libgraphmatch-rem` La razón principal: el FFI rebajaba un CSR completo en cada invocación a través de la frontera, mientras que el path Rust lee `StreamEdge` en place sin copia.

4.6. Dominios de aplicación

La primitiva `intersect\_sorted\_u32\_*` se invoca en tres subsistemas:

- **LFTJ worst-case-optimal** para enumeración de triángulos, K_4, C_6, K_5. Es el uso dominante en workloads con patrones cíclicos. Ver `core/graph-engine/src/lftj.rs` y los benchmarks `benches/triangle\_count.rs`, `benches/triangle\_query.rs`.
- **Anomaly scorer** via `gallop\_ge\_u32` para rank lookups sobre arrays ordenados de scores.
- **Componente de *neighborhood concentration*** en el scoring compuesto, que necesita intersectar vecindarios para medir cuán cerrada queda la sub-red del path.

4.7. Status badge en UI

El frontend Tauri muestra la ISA activa via la cadena `simd\_isa` expuesta por `runtime\_info()`:

$$\underbrace{\texttt{cmd_get_simd_isa}}_{\text{Tauri command}} \rightarrow \underbrace{\texttt{api.runtime_info()}}_{\text{platform/api}} \rightarrow \underbrace{\texttt{simd_rust::isa_label()}}_{\text{engine}} \rightarrow \texttt{"avx2"} \mid \texttt{"scalar"}$$

La detección ocurre por query (no en startup) vía `is\_x86\_feature\_detected!("avx2")`, así que el binario distribuible se adapta a la CPU sin recompilar.

Capítulo 5

DFS temporal guiado por score

5.1. Rol dentro del pipeline

El matcher SIMD (cap. 4) enumera paths *estructuralmente* válidos: cumplen los tipos de nodo/arista y la longitud del patrón. Sobre ese conjunto se aplica una segunda fase, *refinement*, que:

1. impone orden temporal sobre las aristas del path;
2. computa score compuesto por path;
3. poda paths debajo de umbral o que exceden cotas de ramificación (`k-simplicity`).

La fase *refinement* se implementa como un DFS temporal guiado por score parcial. Este capítulo describe el algoritmo, las invariantes que preserva, y por qué usamos DFS en vez de las variantes clásicas (Ullmann, VF2, TurboISO).

5.2. Hipótesis como forma de patrón

Una hipótesis en el DSL tiene forma:

```
User -[Auth]-> Host -[Connect]-> IP {k=3, window=30m, min_score=0.4}
```

que se desazucara a una secuencia ordenada de restricciones:

- tipos por posición $(\tau_0, \tau_1, \tau_2, \tau_3)$;
- tipos de arista por salto (ρ_0, ρ_1, ρ_2);
- ventana temporal W sobre el path completo;
- cota estructural k (rama del DFS);
- filtro post-hoc score $\geq s_{\text{mín}}$.

Definición 5.1 (Path instancia del patrón)**.** Una secuencia $\pi = (v_0, e_0, v_1, e_1, \ldots, v_n)$ es instancia si: (i) $\text{type}(v_i) = \tau_i$ para $i = 0..n$; (ii) $\text{type}(e_i) = \rho_i$ para $i = 0..n-1$; (iii) $t(e_0) \leq t(e_1) \leq \cdots \leq t(e_{n-1})$ y $t(e_{n-1}) - t(e_0) \leq W$; (iv) la ramificación de cada nodo interno del patrón es $\leq k$.

5.3. Algoritmo

Algorithm 5.1: DFS temporal guiado por score parcial

Input: Grafo G, patrón (τ, ρ), ventana W, k, $s_{\text{mín}}$
Output: Paths ordenados por score

$R \leftarrow \emptyset$ ▷ `resultado`
foreach $v_0 \in V : \mathit{type}(v_0) = \tau_0$ **do**
  DFS(v_0, path= $[v_0]$, $t_0 = -\infty$, score= 0, depth= 0);
end
return top N de R por score;

procedure DFS(v, path, $t_{\text{mín}}$, score, depth):
if $\mathit{depth} = n$ **then**
  if $\mathit{score} \geq s_{\text{mín}}$ **then**
    $R \leftarrow R \cup \{(\text{path}, \text{score})\}$;
  end
  return;
end
$\tau_{\text{next}} \leftarrow \tau_{\text{depth}+1}$; $\rho_{\text{next}} \leftarrow \rho_{\text{depth}}$;
$N_v \leftarrow \{(e, u) : e = (v, u), \text{type}(e) = \rho_{\text{next}}, \text{type}(u) = \tau_{\text{next}}\}$;
if $|N_v| > k$ **then**
  return ▷ `poda por k-simplicity`
end
foreach $(e, u) \in N_v$ **do**
  if $t(e) < t_{\text{mín}} \vee t(e) - t(e_0) > W$ **then**
    continue
  end
  $s' \leftarrow$ update_partial_score(score, v, e, u);
  if $\mathit{upper_bound}(s', \mathit{depth} + 1) < s_{\text{mín}}$ **then**
    continue ▷ `poda por score`
  end
  DFS(u, path $\cup\{e, u\}$, $t(e)$, s', depth+1);
end

5.4. Propiedades y complejidad

Lema 5.2 (Orden temporal)**.** *Todo path π emitido por DFS satisface $t(e_0) \leq t(e_1) \leq \ldots$ por construcción (gracias a $t_{\text{mín}}$).*

Lema 5.3 (Monotonía del score parcial)**.** *Si upper_bound(s, d) acota superiormente el score alcanzable extendiendo d hops más, podar cuando upper_bound $< s_{\text{mín}}$ es seguro: ningún path emitido por esa rama superaría $s_{\text{mín}}$.*

La cota superior se construye asumiendo puntajes máximos locales para los componentes que monotonizan (entity rarity, temporal novelty) y cero para los que requieren contexto completo (neighborhood concentration). No es apretada pero sí válida.

La complejidad en el peor caso es $O(|V_{\tau_0}| \cdot k^n)$. En la práctica el DFS emite orden de decenas a miles de paths para hipótesis de 3-4 hops sobre grafos de 10^4-10^5 nodos, con tiempos sub-segundo reportados en benchmarks.

5.5. ¿Por qué DFS y no Ullmann/VF2?

Los clásicos (Ullmann [5], VF2 [6], TurboISO) resuelven *subgraph isomorphism* sobre grafos estáticos con pesos uniformes. Nuestro setup difiere en tres puntos:

1. **Multigrafo temporal**. Múltiples aristas entre un par de nodos distinguidas por timestamp. Ullmann/VF2 asumen único edge.
2. **Score como guía**, no como post-filtro. Queremos podar ramas prometedoramente malas temprano. DFS con *best-first* en la frontera de siblings es natural.
3. **Paths, no subgrafos**. Buscamos caminos lineales (a veces con fork), no cliques ni cualquier subgrafo.

DFS encaja los tres. La complejidad es peor en el peor caso pero la cota k (*k-simplicity*) lo acota estructuralmente y el upper-bound de score lo acota semánticamente.

Capítulo 6

Scoring compuesto

6.1. Motivación

El DFS temporal produce paths estructuralmente válidos. Ninguno está *etiquetado* como malicioso. El scoring convierte el conjunto de paths en una lista ordenada que un analista puede triar. El diseño persigue tres propiedades:

1. **Descomposable**. Un score único monolítico ("la caja negra dijo 0.87") no informa al analista. Queremos que cada path muestre sus componentes y que el analista pueda razonar sobre ellos.
2. **Operable**. Los pesos deben ser ajustables por entorno sin retraining de un modelo. Un analista con contexto organizacional sabe que *novelty temporal* importa más en una telco que en una boutique financiera.
3. **Degradable**. Si un componente no está disponible (típicamente `gnn_threat` sin modelo), el score total debe seguir siendo significativo.

6.2. Los cinco componentes

Todos los componentes retornan $[0, 1]$. Sean v un nodo, e una arista, $\pi = (v_0, e_0, \ldots, v_n)$ un path.

6.2.1. Entity rarity

$$\mathrm{ER}(v) = 1 - \frac{\ln(\mathrm{freq}(v))}{\ln(\max_u \mathrm{freq}(u))}$$

donde $\mathrm{freq}(v)$ es el grado del nodo en el grafo actual. El log normaliza la distribución de grados power-law. Rarity alta para nodos que aparecen poco; rarity 0 para el hub dominante. Aplicado por path como promedio sobre $v_0, \ldots, v_n$.

6.2.2. Edge rarity

Análogo al anterior pero sobre pares $(\mathrm{type}(v), \rho, \mathrm{type}(u))$:

$$\mathrm{EdgeR}(e) = 1 - \frac{\ln(\mathrm{freq}(\mathrm{type}(e)))}{\ln(\max_{\rho'} \mathrm{freq}(\rho'))}$$

Captura qué tan raro es *este tipo de relación* en el grafo. Un `User-[Spawn]->Process` dentro de un grafo de red pura es maxR; dentro de un grafo dominado por endpoint, es común.

6.2.3. Neighborhood concentration

$$\mathrm{NC}(\pi) = 1 - \frac{|\bigcap_{v\in\pi} N(v)|}{|\bigcup_{v\in\pi} N(v)|}$$

Jaccard-complement entre los vecindarios de los nodos del path. Si los vecindarios se solapan mucho (actividad rutinaria en la misma sub-red), NC es bajo. Si los nodos del path se tocan pero pertenecen a mundos distintos (lateral movement real), NC es alto.

6.2.4. Temporal novelty

$$\mathrm{TN}(\pi) = \min_{e\in\pi} \mathrm{novelty}(e), \quad \mathrm{novelty}(e) = 1 - \frac{\mathrm{n_past}(e)}{\mathrm{n_past}(e)+1}$$

donde n_past(e) es la cantidad de veces que se observó el mismo par ordenado (v_u, v_v) en los 30 días previos. Una arista nunca vista antes tiene novelty = 1; una arista observada ayer 10 veces tiene ~ 0.09. El path toma el mínimo porque una sola arista rutinaria basta para hacer el path poco novedoso.

6.2.5. GNN threat

$$\mathrm{GT}(\pi) = \begin{cases} f_\theta(\pi) & \text{si hay modelo cargado} \\ 0 & \text{caso contrario} \end{cases}$$

donde f_θ es un clasificador GNN entrenado sobre paths etiquetados. El "0 si no hay modelo" no penaliza: el compositor combina con peso $w_5 = 0$ por defecto (ver cap. 8).

6.3. Composición

$$S(\pi) = w_1\,\overline{\mathrm{ER}} + w_2\,\overline{\mathrm{EdgeR}} + w_3\,\mathrm{NC} + w_4\,\mathrm{TN} + w_5\,\mathrm{GT}$$

con $\sum w_i = 1$ y pesos por defecto $(0.25, 0.25, 0.30, 0.20, 0.00)$. La ausencia de peso para GNN obedece al principio de *degradación segura*: el sistema debe funcionar en el primer día sin modelo entrenado. Si el cliente entrena uno, sube w_5 a expensas de los otros.

6.4. Propiedades numéricas

Invariante. $S(\pi) \in [0, 1]$ por construcción ya que cada componente $\in [0, 1]$ y $\sum w_i = 1$.

Invariante. Si dos paths π, π' difieren sólo en un nodo $v \neq v'$ con $\mathrm{freq}(v) > \mathrm{freq}(v')$, entonces $\mathrm{ER}(\pi) \leq \mathrm{ER}(\pi')$. *Ceteris paribus*, favorece el path con entidades más raras.

El test `core/graph-engine/tests/anomaly_proptest.rs` verifica estabilidad numérica: pequeñas perturbaciones en el grafo no causan saltos en el score superiores a la perturbación.

6.5. Explicación por path

El endpoint `explain\_hunt\_result` devuelve los cinco componentes normalizados y su producto con el peso. La UI los presenta como barras apiladas. El diseño explicable es una respuesta deliberada a la trampa de los scores opacos: un analista que no entiende por qué un path subió no puede ni confiar ni cuestionar el ranking.

La especificación formal de endpoints y fórmulas vive en el libro *Especificación*.

Capítulo 7

Decisión arquitectónica: eliminación de la frontera FFI

Estado (2026-05-12). Este capítulo documenta la decisión 2026-05-07 de eliminar `libgraphmatch` *y la frontera FFI Rust↔C++ que la sostenía. Reemplaza al capítulo original "FFI con typestate y aislamiento", que describía el diseño typestate+RAII de* `GmGraph/GmMatcher/GmResults`*. Ese diseño nunca fue defectuoso; fue* redundante*: el DFS puro-Rust resultó 3-25× más rápido y la frontera dejó de aportar valor. El contrato C ABI histórico vive en* `docs/\_archive/` *para referencia.*

7.1. Diseño original (resumen breve)

Hasta abril de 2026 el matcher cruzaba FFI a un motor C++ AVX2 (`libgraphmatch`). El cruce estaba blindado por tres herramientas del sistema de tipos de Rust:

- **Typestate** sobre `GmGraph<State>` con `State $\in $ \{Building, Finalized\}`, codificando la máquina de estados "no se puede matchear sobre un grafo todavía abierto a inserciones" en la firma del método.
- **RAII** via `Drop` sobre `GmGraph`, `GmMatcher` y `GmResults`, ligados al lifetime del grafo padre vía `'g`.
- **ABI version check** en load-time: `gm\_abi\_version()` comparado contra `REQUIRED\_ABI`; mismatch de major ⇒ panic temprano (*fail-fast* en lugar de UB silencioso).

Esto materializaba el principio *shearing layers* en el nivel de tipos: el dolor de C++ se quedaba en una sola crate (`core/matcher-ffi`) y el resto del sistema veía un handle Rust opaco.

7.2. Hallazgo empírico que precipitó la eliminación

El 2026-05-07 se midió el costo real del FFI sobre el mismo host (Ryzen 5 PRO 5650U, Zen 3, AVX2). El motor C++ resultó estrictamente más lento que el DFS puro-Rust en *cada* patrón de los benchmarks:

Patrón (big graph 20k)	Rust ms	C++ SIMD ms	speedup
spawn_chain L=2	0.187	3.637	**19.4× Rust**
spawn_chain L=3	0.181	3.459	19.1× Rust
lateral L=3	0.141	3.412	24.2× Rust
lateral L=4	0.160	3.217	20.1× Rust
any_chain L=2	1.265	4.356	3.4× Rust
any_chain L=3	1.256	4.071	3.2× Rust
any_chain L=4	1.562	4.626	3.0× Rust

Tabla 7.1: Baseline comparativo del DFS Rust vs FFI a libgraphmatch sobre el bench `bench\_dfs`. El kernel C++ pierde por factores de 3 a 19. Fuente: `docs/perf/2026-05-07-pre-libgraphmatch-removal.md`.

Análisis post-mortem (resumido del documento de perf):

1. `build\_gm\_graph` reconstruía el CSR C++ *en cada llamada*, copiando los 20k nodos y 160k aristas a través de la frontera. El DFS Rust lee `StreamEdge` en place, sin copia.
2. El DFS Rust tiene prefetch L1 lookahead (cap. 4, sec. 4.2.2) y la cache SoA `entity\_type\_tags` (sec. 4.2.1) que reducen el chequeo de tipo a una probe L1; el matcher C++ hacía `graph\_.node\_type(e.t` escalar por arista.
3. La saturación del cap a 10^4 resultados que ya estaba optimizada en el lado Rust (`HuntResult` con `Vec<Arc<str>>`, shipped 2026-04-30) nunca llegó al lado C++; el matcher allocaba un `MatchResult` fresco (`std::vector<NodeId>` + `std::vector<Timestamp>`) por cada uno de los 10^4 cap entries.

El kernel C++ tenía AVX2 pero no la *usaba* en el DFS: `dfs\_match\_iterative` iteraba aristas salientes secuencialmente, idéntico al DFS Rust. La primitiva SIMD sorted-set sólo se exponía en `simd\_intersect.cpp`, nunca era invocada por el matcher.

7.3. Decisión

Eliminar el árbol C++ completo. Concretamente:

- Borrar `libgraphmatch/` (motor C++ AVX2, ≈ 12 787 LOC, 8 `.cpp`).
- Borrar `core/matcher-ffi/build.rs` y la dependencia `cc-rs`.
- Borrar los tests `core/matcher-ffi/tests/\{equivalence\_proptest, safety\_invariants, simd\_matcher\_test, abi\_mismatch\_test\}.rs`.
- Borrar el feature `simd` de `core/matcher-ffi`, `platform/api`, `apps/tauri/src-tauri` (las tres únicas crates que lo declaraban).
- Reducir `core/matcher-ffi/src/lib.rs` a un *thin forwarder* Rust de ≈30 líneas que delega al engine, manteniendo la firma `search\_temporal\_pattern\_simd` para no tocar call-sites en `platform/api` en el mismo PR.
- Re-wirear el badge de ISA en la UI: `cmd\_get\_simd\_isa` ahora pasa por `platform/api::runtime\_info` → `$graph\_hunter\_core::simd\_rust::isa\_label()`, sin tocar FFI.

7.4. Forma del crate residual

`core/matcher-ffi` sobrevive como un crate Rust puro (no es FFI; el nombre es histórico). Su `Cargo.toml`:

```
[package]
name = "graph_hunter_matcher_ffi"
edition = "2024"

[dependencies]
graph_hunter_core = { path = "../graph-engine" }
mimalloc = { version = "0.1", default-features = false }
```

Ni `cc`, ni `bindgen`, ni `build.rs`. La única función pública es:

```
pub fn search_temporal_pattern_simd(
    hunter: &GraphHunter,
    hypothesis: &Hypothesis,
    time_window: Option<(i64, i64)>,
    max_results: Option<usize>,
) -> Result<(Vec<HuntResult>, bool), GraphError> {
    let scorer_ready = hunter.anomaly_scorer
        .as_ref()
        .map(|s| s.is_finalized())
        .unwrap_or(false);
    if scorer_ready {
        let cap = max_results.unwrap_or(10_000);
        hunter.search_temporal_pattern_smart(hypothesis, time_window, cap)
    } else {
        hunter.search_temporal_pattern(hypothesis, time_window, max_results)
    }
}
```

El crate es candidato a eliminación una vez que los consumers (actualmente sólo `platform/api`) llamen directamente al engine.

7.5. Consecuencia arquitectónica

Tres efectos:

1. **Cero frontera de UB que defender**. Sin `extern "C"`, sin punteros opacos, sin RAII manual sobre handles foráneos. El borrow checker es la garantía completa.
2. **Cero verificación de ABI**. No hay biblioteca nativa dinámica que pueda quedar desincronizada del binario Rust. Distribuir GraphHunter es shippear un único árbol Rust.
3. **Compile time reducido**. `bench\_dfs` pasó de 1 min 13 s (release, fresh target, incluyendo cc-rs sobre 8 `.cpp`) a ∼9 s incremental. El primer build clean de `graph\_hunter\_core` cayó de ∼70 s a ∼50 s.

7.6. Pérdida controlada

El diseño typestate+RAII no fue defectuoso; se eliminó porque la frontera que protegía dejó de existir. Cuatro lecciones que el proyecto conserva:

- **Medir antes de mantener**. El FFI sobrevivía por inercia, no por evidencia. La baseline del 2026-05-07 fue la primera medición rigurosa post-2025 y bastó para desactivar la decisión original.

- **Una frontera FFI cuesta más que su código**. El costo dominante no era la línea de `cc-rs`: era el CSR rebuild por llamada, el alloc por resultado y el corredor sin prefetch del lado C++. Migrar a SIMD nativa Rust eliminó los tres de una.
- **La primitiva SIMD no necesita C++**. La intersección sorted-set AVX2 sobrevive como Rust puro (`simd\_rust.rs`, cap. 4, sec. 4.3) con runtime dispatch y sin perder velocidad.
- **El patrón typestate sigue siendo válido** para cualquier frontera futura que materialmente justifique su existencia. Su uso anterior queda documentado como caso de estudio.

Referencias

- `docs/perf/2026-05-07-pre-libgraphmatch-removal.md` — baseline pre-migración.
- `docs/perf/2026-05-07-post-libgraphmatch-removal.md` — números post-migración y verificación.
- `docs/perf/2026-05-07-libgraphmatch-simd-audit.md` — inventario de primitivas SIMD que el FFI exponía y dónde están hoy en Rust.
- `docs/\_archive/architecture-es/chapters/ch11-simd.tex` — arquitectura SIMD original con motor C++.

Capítulo 8

Model gate: GNN experimental opcional

8.1. El dilema del componente ML

Un clasificador ML dentro de un sistema productivo tiene dos modos de falla sistémicos más allá de su accuracy:

1. **Bloqueo por modelo ausente**. El cliente recién instalado no tiene modelo entrenado; si el sistema exige uno, no puede ni empezar.
2. **Degradación silenciosa**. El modelo produce predicciones sin confianza adecuada; el sistema las integra como si fueran ciertas; el analista las trata como groundtruth.

El patrón *model gate* es la respuesta arquitectónica a ambas. La funcionalidad ML es opcional por contrato, no por configuración.

8.2. Contrato

El clasificador vive en `core/gnn/` detrás de un trait:

```
pub trait ThreatClassifier: Send + Sync {
    fn classify_path(&self, path: &[NodeRef], edges: &[EdgeRef]) -> f64;
    fn model_info(&self) -> ModelInfo;
}

pub struct NoModel;
impl ThreatClassifier for NoModel {
    fn classify_path(&self, _: &[NodeRef], _: &[EdgeRef]) -> f64 { 0.0 }
    fn model_info(&self) -> ModelInfo { ModelInfo::none() }
}

pub struct OnnxClassifier { session: OrtSession, ... }
impl ThreatClassifier for OnnxClassifier { ... }
```

El composer de scoring no conoce cuál está activo. Recibe un `Arc<dyn ThreatClassifier>`. Si la configuración no apunta a un modelo ONNX, se instancia `NoModel` y devuelve 0. `w\_5` es 0 por defecto: multiplicado por 0 no penaliza.

8.3. Arquitectura del modelo v2

El modelo actual (*v2 experimental*) es una GraphSAGE de dos capas sobre el path instanciado. Entrada: features por nodo (type one-hot, grado log-normalizado, first-seen-age) y por arista (type one-hot, timestamp delta al anterior). Salida: probabilidad de clase "threat".

Entrenamiento fue sobre un dataset etiquetado de logs Sentinel AAD con incidentes reales etiquetados por un analista senior (∼2000 paths positivos, ∼20000 negativos sampleados). Export a ONNX vía `tch-rs`. Carga en runtime con `ort`.

8.4. Separación layer-wise

El modelo ML vive en el *core* (`core/gnn`), consume tipos del grafo pero no conoce la superficie HTTP/MCP. El switch entre `NoModel` y `OnnxClassifier` está en un módulo de configuración (`core/graph-engine/src/scoring/gate.rs`). La decisión se documenta en ADR-003.

8.5. Honestidad epistémica

- El modelo *no está entrenado* para FortiAnalyzer, Cognito ni Zoho. Tirarlo sobre esos datasets sin retraining da scores no informativos. Por eso el peso por defecto es 0.
- El modelo *no es confiable como único juez*. Sirve como un quinto voto en la composición, no como clasificador autónomo.
- Un modelo por cliente / por vertical es probablemente el camino. Esto es una observación, no una decisión tomada.

8.6. Extensiones previstas

- Exposición del vector de confianza (no sólo el score), útil en UI para distinguir "GNN está seguro" de "GNN no sabe".
- Training pipeline reproducible (`training/`) ya scaffoldea pero está fuera del alcance de este reporte.
- Model gate per-vertical: selección del ONNX por cliente/contexto, no global.

Capítulo 9

Ingestor agéntico: LLM-as-compiler offline y drift-as-signal

9.1. Problema

La ingesta productiva de logs tiene dos propiedades en tensión: debe ser *determinística* (dos corridas del mismo batch producen la misma salida, byte-a-byte) y debe ser *adaptable* (formatos nuevos aparecen, y los existentes evolucionan aguas arriba sin aviso). La práctica dominante resuelve la primera y abandona la segunda: un parser hardcodeado por formato, release del daemon por cada cambio, parser genérico heurístico como red de seguridad con pérdida de fidelidad silenciosa.

GraphHunter resuelve las dos simultáneamente separando el sistema en tres carriles con perfiles distintos de determinismo, latencia y quién los opera. El presente capítulo describe el diseño, la matemática del contrato, y qué asume sobre el humano en el loop.

9.2. Decisión: tres carriles

- **Fast lane**. Un compilador determinístico compile : FieldConfig $\to$ VRL emite programas textuales con identidad estable $\text{hash}(f) = \text{sha256}(\text{compile}(f))$. El runtime ejecuta VRL por evento. Cero LLM, cero red, cero nondeterminismo. El hash se estampa en cada evento canonical (`provenance.mapping_hash`), cerrando la trazabilidad dato$\leftrightarrow$lógica.
- **Slow lane**. Un LLM actúa como *compilador asistente* offline: dado una muestra del formato y un target canonical (OCSF v1.4), propone un `FieldConfig`. El output del LLM son datos, no código. El código lo produce el compilador VRL determinístico a partir de esos datos.
- **Learning lane**. El fast lane emite métricas estructurales baratas por batch (histogramas, cardinalidades, ratios DLQ). Un monitor de drift compara contra el perfil esperado del mapping activo; cuando supera umbral, abre re-review en el slow lane sin detener la ingesta.

El contrato entre carriles es un par $(\text{FieldConfig}, \text{OCSFMapping})$: ambos datos, ambos versionados. Ningún carril llama a otro por función en tiempo real. El acoplamiento es por artefacto.

9.3. LLM como compilador, no como traductor

La literatura de ingesta con LLM se suele deslizar a una variante riesgosa: el LLM procesa eventos en caliente. Esto falla de manera predecible por tres razones:

1. **Latencia**. Un LLM remoto agrega cientos de ms por request. Un LLM local agrega decenas de ms. Para un stream de miles de eventos por segundo, ninguno alcanza.

2. **Costo**. Por evento ingerido pagás un token-count. A escala SIEM (millones de eventos/día), este costo es órdenes de magnitud mayor que el compute determinístico.

3. **Superficie de falla**. Un LLM que parsea un batch mal aplica el error a todos sus eventos. Sin gate humano, ese error entra al grafo y contamina todo hunt posterior.

La decisión de diseño: el LLM no ve eventos productivos. Ve muestras explícitas que el operador o el monitor de drift le entregan. Lo que produce son *datos que describen una transformación* (el FieldConfig), no la transformación ejecutada. La ejecución la hace un compilador determinístico que no es un LLM.

Esto es análogo a cómo un compilador optimizador corre offline y produce código que un runtime simple ejecuta en caliente. Aquí el compilador offline es un LLM asistido por humano; el runtime caliente es el VRL interpreter.

9.4. Drift como señal, no como error

Un cambio aguas arriba (nuevo campo, separador modificado, semántica movida) es, desde la perspectiva del ingestor, un cambio en la distribución empírica de la entrada. Si lo tratamos como *error*, las opciones son dos: parar la ingesta (inaceptable en producción), o ingerir basura (inaceptable para análisis).

Lo tratamos como *señal*. Operacionalmente:

- Cada batch B genera un vector empírico $\hat{P}(B) \in \mathbb{R}^k$ de estadísticas (histograma por tipo, cardinalidad por campo, ratio DLQ, distribución de timestamps).
- El mapping m en uso tiene un perfil esperado P_m calculado sobre su ventana de calibración.
- Una prueba Kolmogorov-Smirnov por campo, agregada por max-divergencia, produce $d(B, P_m)$.
- Si $d(B, P_m) > \tau$, se abre una *propuesta de re-review* — entrada en la cola del slow lane con el batch anómalo adjunto. El fast lane sigue ingestando B con el mapping m.
- El humano, con ayuda del LLM, decide si el mapping debe evolucionar. En el caso afirmativo, publish_mapping emite un nuevo m' con hash distinto; el fast lane recoge m' en el siguiente batch.

Esto invierte el esquema tradicional. En lugar de parar para aprender, aprendemos mientras seguimos ingestando. El costo es que durante el período de drift detectado pero no aprobado, el grafo recibe eventos con mapping potencialmente degradado; la mitigación es que τ se configura bajo y la ventana de revisión es minutos-horas, no días.

9.5. Propiedades formales

Sea $f \in$ FieldConfig y e un evento crudo. Escribamos $\text{run}(f, e) = \text{vrl_run}(\text{compile}(f), e)$ para la salida canonical del fast lane.

Determinismo. Para todo f, e: $\text{run}(f, e)$ depende sólo de f y e. La misma entrada produce idéntico output en corridas distintas y máquinas distintas.

Trazabilidad. Cada evento canonical lleva $\text{hash}(f)$ en su campo `provenance.mapping\_hash`. A partir del hash se recupera f desde la mapping library; a partir de f se reconstruye $\text{compile}(f)$; ejecutar sobre el evento crudo original reproduce el canonical.

Gate humano. Toda transición $m \to m'$ en la mapping library es mediada por aprobación humana (`agentic_review_approve`). No hay camino automático LLM→productivo.

No-block. El learning lane no detiene el fast lane. Si el LLM falla, se cae la red, o el humano tarda, el fast lane sigue ejecutando el m vigente.

Las cuatro propiedades son arquitectónicas, no validadas empíricamente en este reporte. La evaluación cuantitativa (latencia del fast lane comparada con parsers hardcoded, accuracy del LLM en inferir FieldConfig, tasa de verdaderos/falsos positivos del drift monitor) es trabajo futuro y se reporta en capítulos separados cuando haya números.

9.6. Trabajo relacionado

El uso del LLM como compilador asistido tiene precedentes en *code synthesis* (Copilot, Codex) y *schema matching* con transformers. La diferencia aquí es que el compilador humano-LLM produce *datos tipados* (`FieldConfig`), no código de programa; y la ejecución en caliente es determinística sobre esos datos.

Drift detection sobre streams tiene tradición en machine learning de producción (ADWIN, Page-Hinkley). El aporte de GraphHunter es aplicar estas métricas no a predicciones sino a *forma estructural* (histogramas de tipo, cardinalidades) como proxy barato de cambio de schema.

OCSF como target canonical es adopción de un estándar externo; el aporte de GraphHunter es usarlo como *lingua franca* entre mappings heterogéneos y el motor de grafo.

9.7. Limitaciones y trabajo futuro

- El fast lane usa un intérprete VRL embebido; la sustitución por la crate `vrl` upstream es follow-up de performance.
- No hay evaluación cuantitativa aún. Los números de latencia, accuracy LLM, y drift precision/recall se reportan cuando el sistema lleve tiempo suficiente en producción.
- El umbral τ de drift es global; idealmente sería por-campo y auto-ajustable a partir de señal de false-positive.
- El gate humano es el ancla de seguridad; también es el cuello de botella. Optimizar la UX de review agéntica es trabajo de producto, no de arquitectura.

Capítulo 10

Evaluación

La evaluación se organiza en tres ejes: (i) benchmarks cuantitativos de latencia y throughput, (ii) property-based tests que verifican invariantes, (iii) smoke end-to-end sobre datasets sintéticos. Cada eje responde a una pregunta distinta.

10.1. Benchmarks de performance

Los benchmarks viven en `core/graph-engine/benches/` y se corren con *criterion*. Reportamos números medidos con *baseline* `pre-l` (cierre del *M-track* de abril 2026) sobre un host de desarrollo (Zen 3 / AVX2, *thin LTO*, sin `target-cpu=native` — ese flag regresiona en CPUs híbridas, ver `docs/perf/release-profile.md`). Mediana sobre 100 muestras criterion, 2 s *warm-up*, 5 s *measurement*.

Benchmark	*Pre-LTO*	*Pre-L*	*Mejora*
Hunt 1-hop, $\|V\| = 10^3$	302 µs	97.6 µs	3.1×
Hunt 1-hop, $\|V\| = 10^4$	3.4 ms	874 µs	3.9×
Hunt 1-hop, $\|V\| = 10^5$	40.2 ms	**13.5 ms**	3.0×
Match 2-hop, $\|V\| = 10^4$	—	4.46 ms	—
Match 3-hop, $\|V\| = 10^4$	—	10.23 ms	—
Match 4-hop, $\|V\| = 10^4$	56 ms	**20.36 ms**	2.7×
Ingest sysmon NDJSON, 1k	5.4 ms	4.74 ms (210K ev/s)	1.1×
Ingest sysmon NDJSON, 10k	66 ms	38.79 ms (250K ev/s)	1.7×
Ingest sysmon NDJSON, 50k	327 ms	204.4 ms (240K ev/s)	1.6×
Score path (depth 4)	1.04 µs	401 ns	2.6×

Tabla 10.1: Latencia y *throughput* sobre las cinco *suites* canónicas *matcher* y la *suite* de *ingest*. La columna *pre-LTO* es el *baseline* previo a la optimización de *profile* y al *M-track* (*S3, M1, M2, M3* y refinamientos M3+/M3++); la columna *pre-L* es el estado al cierre de abril 2026. Datos completos en `docs/paper/data/bench-table.md`, generado por `scripts/bench-report.sh`.

Operadores canónicos WCOJ. El *M-track* agregó cuatro *count operators* con *symmetry-breaking hardcoded* para las *shapes* estructuralmente conocidas (K_3, K_4, K_5, C_6). En el extremo denso del *sweep ER* ($V = 1000$, $E = 9913$, $C_6 = 926763$ *instances*), *naive* $O(m \cdot d^4 \cdot \log d)$ tarda 2.03 s, mientras que la *canonical+par* con *star-from-min* $a < \{b, c, d, e, f\}$ sobre $\mathbb{Z}/6$ tarda 22.5 ms — $\sim 90\times$. Para $K_3/K_4/K_5$ el corte respectivo es 11–12× sobre *naive* con *chain* $a < b < c[< d[< e]]$. Detalle en `docs/perf/latex/cambios-perf-2026-04.tex` (capítulos M3 y M3++).

Observaciones:

- Hunt 1-hop a 10^5 nodos en 13.5 ms ronda el *abstract claim sub-10 ms* de *Black Hat Arsenal 2026* con un *gap* restante de $\sim 35\,\%$ (cubrible por la C-tier *frontier-batched + AMAC*, ya presente bajo *feature flag*).

- *Ingest* a 250K ev/s sigue 4× bajo el *target* de 1M ev/s. La *lift* restante depende de *simd-json* con *schema-known fast-path*, no incluido en el *M-track*.
- La composición de score es barata: escala linealmente con la longitud del path. La *depth 4* cae 2.6× por la migración `HuntResult: Vec<String>` → `Vec<Arc<str>>`, que evita path.len() · String::from(…) *allocations* por *result*.

10.2. Property-based tests

Tres propiedades clave se testan con *proptest*:

1. **Equivalencia AVX2↔escalar de la intersección sorted-set**. Los 15 unit tests de `core/graph-engine/src/simd\_rust.rs` verifican que `intersect\_sorted\_u32\_count` devuelve el mismo conteo que el merge escalar y que una referencia `BTreeSet` sobre slices aleatorios (LCG-deterministic). Paridad LFTJ *naive* = *LFTJ* = *LFTJ+SIMD* en `core/graph-engine/src/lftj`. El proptest histórico de equivalencia FFI `core/matcher-ffi/tests/equivalence\_dfs\_simd.rs` fue eliminado junto con el árbol C++ (2026-05-07).
2. **DSL roundtrip**. `platform/dsl/tests/roundtrip.rs` genera ASTs aleatorios, los imprime como texto DSL, los re-parsea, y verifica igualdad. 10^4 casos por corrida.
3. **Estabilidad numérica del score**. `core/graph-engine/tests/anomaly_proptest.rs` perturba el grafo (agrega/remueve una arista) y verifica que los scores en el nuevo grafo difieren por menos de una cota proporcional al tamaño de la perturbación.

Estos tests capturan clases enteras de bugs que no caen en el radar de tests unitarios con ejemplos fijos.

10.3. Smoke end-to-end manual

Sobre los datasets de `demo_data/` (ver libro *Manual*, apéndice C) se verifica el flujo completo en cada release candidate:

1. Crear sesión, ingerir `fortigate_sample.csv`. Verificar: 50k eventos procesados, ≤50 en DLQ, schema visible en catálogo.
2. Correr `IP -[Connect]-> IP -[Connect]-> IP {k=3, window=30m}`. Verificar: ≥3 paths con score >0.6, `explain\_hunt\_result` devuelve los 5 componentes.
3. Expandir un nodo sospechoso. Verificar pestañas timeline, events, properties pobladas.
4. Taggear path como IOC. Exportar IOCs en formato Sentinel. Verificar JSON con estructura `ThreatIntelligenceIndicator`.
5. Cerrar y reabrir la sesión. Verificar: todo el estado persistido (grafo, tags, notas, hunts históricos).

El smoke está documentado en `docs/_archive/phase1/MIGRATION_PLAN.md`; sobrevive como checklist operativo de release.

10.4. Métricas del proyecto

Tamaño y complejidad del codebase:

Métrica	Valor
Crates Rust en `core/`	4
Crates Rust en `platform/`	9
Crates Rust en `apps/`	3
Crate TypeScript (MCP)	1 (55 tools, 10 dominios)
Tests unitarios + integración	∼450
Property-based tests	3 suites (equiv, DSL, anomaly)
Benchmarks	4 (hunt, dedup, score, ingest)
ADRs	7

Tabla 10.2: Inventario del proyecto a fecha de corte.

10.5. Comparación pre/post shearing-layers

Pre-rediseño, el grafo de dependencias tenía 19 aristas de "capas" cruzando sentidos mezclados, con al menos dos ciclos frágiles entre parsers y el motor de matching. Post-rediseño hay 0 ciclos y 2 aristas residuales `core`→`platform` (ver cap. 11). El *cargo check* del workspace pasó de ∼2m15s a ∼1m05s en frío en el mismo hardware.

El costo directo del rediseño fue un sprint de tres semanas (Fases 2+3+4 cerradas). El costo evitado es difícil de cuantificar: cada feature nueva que no toca el core se paga sola en ahorro de builds y revisiones.

Capítulo 11

Limitaciones

Un reporte interno responsable declara lo que *no* funciona con tanta claridad como lo que funciona. Este capítulo lista las limitaciones conocidas, agrupadas por ámbito.

11.1. Arquitectura

Dos aristas residuales core→platform

El modelo shearing-layers exige que `core` no dependa de `platform`. Quedan dos violaciones:

- `core/graph-engine` depende de `platform/sources` para el adapter que hidrata el grafo desde una fuente canonical. Lo correcto sería invertir: un trait `CanonicalSource` en `core` y la implementación en `platform`.
- `core/graph-engine` depende de `platform/dsl` para la representación estructural de una hipótesis. Idem: un AST mínimo en `core` y el parser en `platform`.

Ambas fueron conservadas durante el rediseño por costo de cambio relativo bajo y alcance creciente del plan. Están listadas en `docs/_archive/phase2/FOLLOWUPS.md` como follow-ups F2.11 y F2.12.

11.2. Modelo ML

GNN no entrenado para FortiAnalyzer

El clasificador v2 fue entrenado sobre Sentinel AAD. Datasets de firewall (FortiAnalyzer), IAM cloud (Cognito) y SaaS (Zoho) carecen de modelo propio. El comportamiento es correcto (retorna 0 con peso 0, no degrada), pero significa que GraphHunter hoy no aporta el quinto componente de scoring en la mayoría de deploys. Training pipeline reproducible queda como trabajo futuro.

Calibración de pesos

Los pesos por defecto $(0.25, 0.25, 0.30, 0.20, 0.00)$ son una elección informada por inspección manual sobre datasets sintéticos. No hay evaluación cuantitativa contra un set etiquetado grande. Un analista con contexto organizacional puede ajustar, pero no hay método automático de búsqueda de pesos óptimos.

11.3. Tests

Flakes conocidos

Tres tests presentan *flakiness* documentada en `docs/_archive/phase1/RISKS.md`:

- Tests de paridad LFTJ *naive* = *LFTJ* = *LFTJ+SIMD*: timeout intermitente con seeds que generan grafos con hubs de grado >5000 (el path *naive* domina; el path SIMD se mantiene rápido).
- Tests del MCP contra daemon: racing en arranque si el daemon tarda más de lo habitual.
- Smoke integración con GeoIP: depende de presencia del archivo MaxMind; falla silenciosa si no está.

No hay fix permanente aún; los dos primeros se mitigan con retry-on-timeout en CI, el tercero con skip condicional.

11.4. Evaluación

Sin benchmarks comparativos formales

No se ejecutaron benchmarks comparativos formales contra SIEMs comerciales (Splunk, KQL, EQL). El hecho de que las superficies sean distintas (tablas vs. grafos) hace la comparación no trivial metodológicamente. Queda como trabajo futuro diseñar un harness que exprese las mismas preguntas en ambos paradigmas y compare tiempo-al-primer-resultado.

Datasets sintéticos de tamaño limitado

Los smoke tests usan datasets <100k eventos. El comportamiento a 10^7 eventos (régimen productivo real) no está medido sistemáticamente. Observaciones empíricas sobre grafos reales sugieren que el matcher escala; no reportamos números porque no son reproducibles.

11.5. Superficies

DSL sin operadores probabilísticos

El DSL expresa patrones determinísticos ("hay una arista"). No expresa "hay una arista que ocurre menos del 5 % del tiempo" o "esta subsecuencia es inusual". Esas preguntas se responden via scoring post-hoc, no via el DSL. Un DSL con cuantificadores probabilísticos es posible pero fuera de alcance.

Canonical OCSF incompleto

El mapping a OCSF cubre los campos comunes (`time`, `actor`, `src\_endpoint`, `dst\_endpoint`, `observables`). No cubre todas las clases (`class\_uid` de OCSF v1.4 excede lo mapeable con el abanico actual de parsers). La extensión por clase va bajo demanda.

Provenance en memoria, no en disco

El `provenance.mapping_hash` de un evento vive con el evento en el grafo, pero la tabla `mapping_hash$\to $VRL source` se regenera al arrancar. Si la `mapping_library` se pierde, los hashes en el grafo quedan colgando. Una persistencia a disco de la library es follow-up conocido.

11.6. Operación

No hay multi-tenant

El daemon asume un solo operador. No hay aislamiento por usuario ni políticas de acceso a sesiones. Para deploys multi-cliente hace falta una capa de gateway externa.

Sin backup automático

La sesión persiste on-close pero no hay snapshotting periódico. Un crash pierde el estado desde el último close.

Capítulo 12

Mapa *claim* → evidencia

Este capítulo condensa cada afirmación técnica del reporte en una única tabla con puntero a la evidencia reproducible. La regla editorial que aplica al paper —"cada número viene de un fixture o de `target/bench-results`"— se ancla aquí.

12.1. Tabla principal

Claim	Evidence	Artifact	Status	Limitation
DSL v1 estable	Roundtrip proptest (256 casos, incl. bindings, HAVING, WITHIN) + semantics doc	`platform/dsl/tests/`, `docs/spec/dsl-v1-semantics.md`	stable	v2 experimental detrás de feature flag
FFI removido (sin frontera que defender)	`libgraphmatch` eliminado el 2026-05-07; `core/matcher-ffi` reducido a ≈30 LOC Rust puro	`docs/perf/2026-05-07-ship-libgraphmatch-removal.md`	shipped	Chasm de estudio del patrón typestate vive en `docs/\_archive/`
AVX2 ≡ escalar (`intersect\_sorted\_u32`)	15 unit tests + LCG-random parity + paridad LFTJ *naive* = *LFTJ* = *LFTJ+SIMD*	`core/graph-engine/src/simd\_rust.rs`, `core/graph-engine/src/lftj.rs`	stable	Sólo verificado en x86_64 AVX2
DFS ≡ DFS smart	Tests de paridad sobre scorer finalizado en `core/graph-engine`	`core/graph-engine/src/graph.rs` (tests integrados)	stable	Hipótesis predicate-free para la equivalencia exacta
Scoring sin GNN por default	$w_5 = 0$ por defecto + tests de independencia y reproducibilidad bit-for-bit	`core/graph-engine/src/tests.rs` (`scoring\_v1\_*`), `docs/spec/scoring-v1.md`	stable	Pesos no calibrados contra dataset etiquetado grande
Layering *core→platform* acotado	`scripts/check-layering.sh` + whitelist explícita en CI	`scripts/check-layering.sh`, `docs/arquitectura/layering-exceptions.txt`	stable	2 aristas legítimas whitelisted (ADR abierta para cierre)
Ingesta canónica 3 fuentes	18 fixtures (6 variantes × Sysmon/Sentinel/Forti) validan OCSF v1.4	`platform/canonical/tests/fixtures/`, `platform/canonical/src/ingest\_sources.rs`	stable	Jerarquía de severity VRL vs legacy pendiente hasta runtime VRL embebido
DLQ reingest funcional	Test roundtrip: config mal → DLQ → fix config → reingest → DLQ vacía	`platform/api/tests/dlq\_smoke.rs`, `platform/canonical/src/reingest.rs`	stable	UI de reingest en Tauri pendiente
3 escenarios E2E	`scripts/demo.sh` + snapshot tests con tolerancia ±10 %	`platform/api/tests/demo\_scenarios.rs`, `scripts/demo.sh`	stable	Fixtures sintéticos; datasets reales bajo NDA
Benchmarks reproducibles	5 suites criterion → JSON → tabla MD	`core/graph-engine/benches/*.rs`, `scripts/bench-report.sh`, `docs/paper/data/bench-table.md`	stable	Números de |V|=100k sólo con `BENCH\_LARGE=1`
Feature flags auditables	Tabla canónica de 14 flags con estado	`docs/spec/features.md`	stable	—
GNN v2 (model gate)	ADR-003 + gate runtime que rechaza synthetic-only	`core/gnn/src/common/model\_gate.rs`	experimental	Modelo no entrenado para FortiAnalyzer/Cognito/Zoho
LLM-as-compiler (VRL)	Validador post-hoc + xgrammar CFG sobre candle (Phi-3-mini-Q4, CPU local)	`platform/constrained-decoder/`, `platform/local-llm/`	experimental	Backend candle y CFG en construcción (M12.2–M12.3); fallback a heurística

12.2. Reclasificación de contribuciones

La v1 del paper listaba el GNN como contribución *validada empíricamente*. Tras el ciclo 2026-04 esto se rebalanceó:

- **Contribución *arquitectónica* (mantenida)**: ADR-003 describe un gate opcional con criterio de admisión explícito; GraphHunter queda listo para absorber modelos entrenados sin re-diseñar el scoring.
- **Contribución *empírica* (retirada)**: no se reclama mejora cuantitativa end-to-end atribuible al GNN mientras no exista un modelo entrenado contra datasets de las tres familias de logs que GraphHunter cubre (Sysmon/Sentinel/Forti). Los números del paper provienen exclusivamente de los cuatro componentes con $w_5 = 0$.

12.3. Roadmap 30/60/90 (desde 2026-04-24)

2026-05-24 (+30 días)

- Cerrar las dos aristas residuales `core` → `platform` (via crate `core/ingest-types` o inversión de trait).
- Entrenar GNN v2 sobre corpus Sysmon open-source (MORDOR, SOC data).
- UI de triage DLQ en Tauri con drill-down a `raw\_event` y `failure\_kind`.

2026-06-24 (+60 días)

- Runtime VRL embebido → test de parity Jaccard ≥ 0.95 entre VRL y `ConfigurableParser`.
- Backend local-llm real con modelo cuantizado que cumple latencia < 5s para el *review loop* agentic.
- Spike de `iceberg` con CI Windows estable.

2026-07-23 (+90 días)

- DSL v2 como stable: registries con modo `strict-unknown`, predicados CEL-like, HAVING con agregaciones ricas.
- Evaluación cuantitativa contra un SIEM comercial en *tiempo al primer resultado* para 3-5 hipótesis canónicas.
- Multi-tenant: gateway externo + aislamiento por sesión/usuario.

12.4. Regla de cierre

Una *claim* sólo entra en la tabla de este capítulo cuando (a) hay un artifact concreto y (b) ese artifact corre verde en CI. Un reclamo pendiente de evidencia vive en 11, no aquí. La divergencia entre ambos capítulos es el síntoma que dispara el próximo ciclo de trabajo.

Capítulo 13

Conclusiones

13.1. Síntesis

GraphHunter parte de una observación simple: la caza de amenazas es naturalmente relacional y los SIEMs la forzan a ser tabular. La arquitectura resultante no es un producto aislado; es la materialización de cinco disciplinas aplicadas simultáneamente:

1. *Shearing layers.* Las capas de cambio rápido no inyectan presión sobre el núcleo. El mismo motor sirve hoy y servirá mañana con distintas superficies.
2. *Correctness por construcción.* La intersección SIMD AVX2 se verifica contra su contraparte escalar en unit tests y paridad LFTJ. La frontera FFI histórica (typestate + RAII + ABI check) fue eliminada el 2026-05-07 al perder su sujeto. Los scores viven en $[0, 1]$ por invariante.
3. *Descomposición explicable.* El score de un path es la suma ponderada de cinco señales que un analista puede inspeccionar una por una. No hay caja negra.
4. *Degradación segura.* El GNN es opcional. La pérdida de conectividad a un LLM remoto no afecta la ingesta productiva. El sistema funciona en el primer día sin ajustes.
5. *Determinismo con adaptación.* El ingestor agéntico coloca al LLM offline, produciendo datos (`FieldConfig`), no código. El código lo produce un compilador determinístico. El humano aprueba, el drift dispara re-review.

13.2. Qué aprendimos del rediseño

Tres lecciones merecen ser preservadas:

- **El costo de una arista incorrecta es largo**. Los dos `core`→`platform` residuales cuestan cada feature nueva. Invertir el tiempo en eliminarlos antes habría ahorrado horas acumuladas.
- **Los tests de equivalencia valen más que los de ejemplo**. La equivalencia AVX2↔escalar de la intersección sorted-set y la paridad LFTJ *naive* = *LFTJ* = *LFTJ+SIMD* atraparon varios bugs sutiles que ningún test unitario tradicional habría expuesto.
- **El LLM como compilador, no como traductor**. Poner al LLM offline y detrás de un gate humano fue la decisión que destrabó la ingesta adaptativa sin sacrificar determinismo.

13.3. Trabajo futuro

Cortes concretos, alcance de un sprint

- Eliminar las dos aristas residuales `core`→`platform` (F2.11, F2.12).
- Persistir `mapping_library` a disco con compactación y retención configurable.
- Training pipeline reproducible para GNN por vertical.

Investigación, alcance mayor

- Benchmarks comparativos formales contra SIEMs comerciales con harness compartido.
- Operadores probabilísticos en el DSL (cuantificadores "esta arista es inusual") sin perder la composicionalidad.
- Aprendizaje en línea de pesos de scoring a partir de feedback del analista (tag IOC → reweighting).
- Multi-tenant con aislamiento por sesión y políticas de acceso.

13.4. Cierre

La apuesta detrás del diseño es que razonar sobre grafos de actividad es la forma natural de investigar incidentes, y que un sistema que respeta esa forma gana en velocidad analítica lo que pierde en familiaridad con los paradigmas tabulares. Los próximos seis meses de uso en producción dirán si la apuesta paga.

Bibliografía

[1] S. M. Milajerdi et al. «HOLMES: Real-Time APT Detection through Correlation of Suspicious Information Flows». En: *IEEE Symposium on Security and Privacy*. 2019.

[2] S. M. Milajerdi et al. «POIROT: Aligning Attack Behavior with Kernel Audit Records for Cyber Threat Hunting». En: *ACM CCS*. 2019.

[3] B. Perozzi, R. Al-Rfou y S. Skiena. «DeepWalk: Online Learning of Social Representations». En: *ACM SIGKDD*. 2014.

[4] Stewart Brand. «How Buildings Learn: What Happens After They're Built». En: *Viking Press* (1994). Origen del modelo de *shearing layers*.

[5] J. R. Ullmann. «An Algorithm for Subgraph Isomorphism». En: vol. 23. 1. 1976, págs. 31-42.

[6] L. P. Cordella et al. «A (Sub)graph Isomorphism Algorithm for Matching Large Graphs». En: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26.10 (2004), págs. 1367-1372.